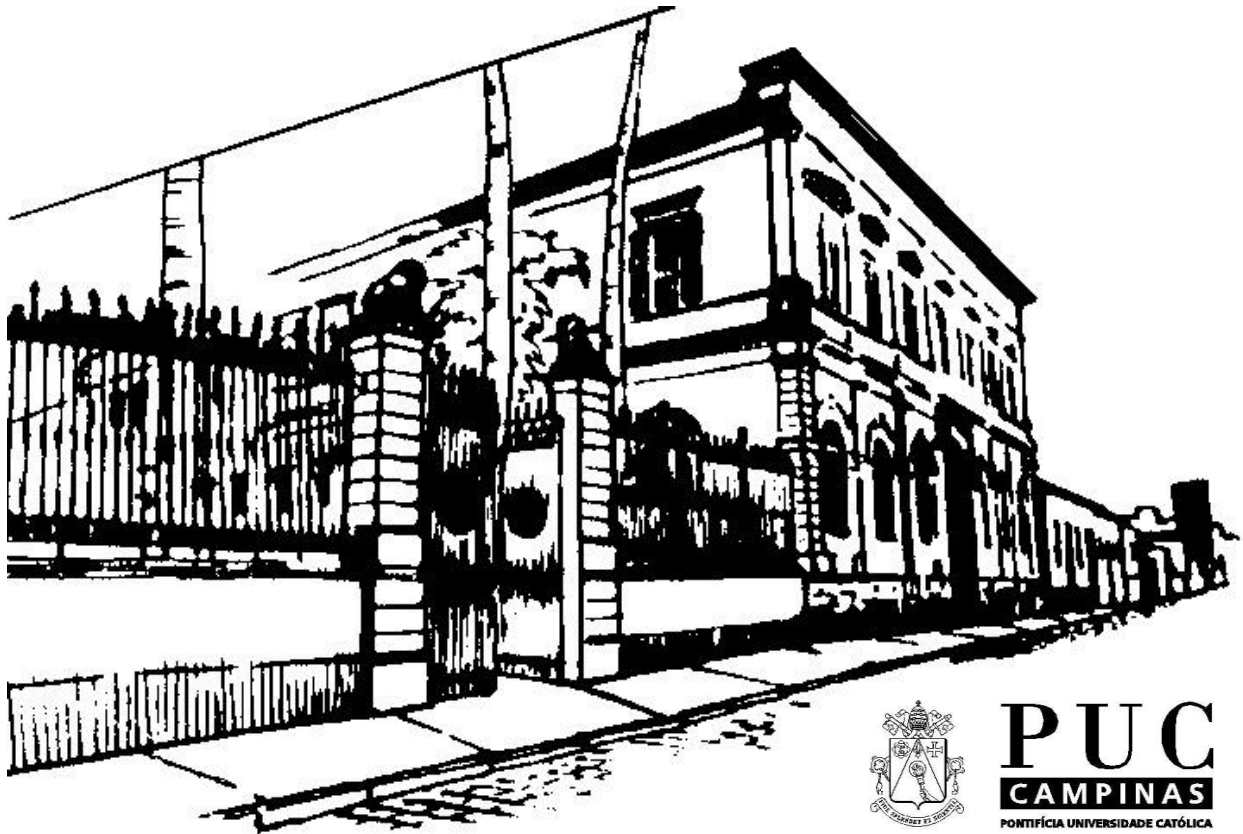




Escola Politécnica da PUCC

*Faculdade de Análise de Sistemas
Curso de Engenharia de Software*

Tópicos de Tecnologia e de Programação

**1º Semestre de 2026
Volume 2**

Prof. André Luís dos R.G. de Carvalho

Índice

ÍNDICE	2
CAPÍTULO III: O PARADIGMA DE ORIENTAÇÃO A OBJETOS	3
INTRODUÇÃO.....	4
MOTIVAÇÃO	4
INSTÂNCIAS.....	5
CLASSES	6
OBJETOS	6
MEMBROS DE CLASSE E DE INSTÂNCIA	7
ENCAPSULAMENTO	7
HERANÇA.....	7
POLIMORFISMO.....	8
SOBRECARGA.....	9
ASSINATURA.....	9
HERANÇA MÚLTIPLA	9
CLASSES ABSTRATAS	10
CONCLUSÃO.....	10
REFERÊNCIAS.....	11
BIBLIOGRAFIA	12

Capítulo III:

O Paradigma de

Orientação a Objetos

Introdução

Apesar de ser um paradigma relativamente novo, o paradigma de orientação a objetos já pode ser considerado um clássico.

Seu uso vem aceleradamente aumentando com o tempo, tendo se firmado praticamente como um “modismo” dos dias atuais. Parece a concretização da previsão anunciada em [Takahashi88], que dizia que a orientação a objetos viria a ser nos anos 80 e 90 aquilo que programação estruturada foi nos anos 70.

Como praticamente todos os paradigmas de programação, o paradigma de orientação a objetos também surgiu juntamente com o desenvolvimento de uma linguagem de programação.

A primeira linguagem a implementar conceitos de orientação a objetos foi a linguagem Símula, proposta por Dahl e Nygaard em 1966. Posteriormente o conceito foi refinado e desenvolvido na linguagem SmallTalk, proposta por Alan Kay em 1972.

A linguagem SmallTalk não somente norteou o desenvolvimento do paradigma, mas, ainda hoje, é considerada a linguagem que suporta de forma mais completa e pura os seus conceitos.

Motivação

Podemos dizer que um programa de computador implementa uma solução computacional para um problema do mundo real.

Assim sendo, construir um programa de computador envolve vários processos de abstração e concretização entre dois mundos, o real (onde existe o problema, a solução, e o procedimento para se chegar do problema à solução) e o virtual (onde existe uma abstração do problema, uma abstração da solução, e um procedimento abstrato para chegar de uma na outra).

Naturalmente, existe uma distância conceitual inevitável entre estes dois mundos. Diminuir esta distância é justamente uma das principais propostas deste paradigma.

Para tanto, o paradigma parte da premissa de que o mundo real não é composto nem por dados (como pretendem os paradigmas orientados a dados), nem por processos (como pretendem os paradigmas orientados a processos), e sim por entidades semi-autônomas que trabalham e interagem cooperativamente.

Tais entidades são os elementos constituintes fundamentais do paradigma e recebem o nome de objeto.

Instâncias

Todo instância possui um estado interno para registrar e se lembrar do efeito de sua operação. O estado interno de uma instância é representado por uma área de memória local ao instância, que é inacessível e indevassável.

Todo instância possui ainda um comportamento, que é representado por um repertório de operações que o instância dispõe para responder a mensagens externas ou mesmo internas à própria instância.

O resultado da operação de uma instância depende não só da mensagem que ele recebeu, mas também de seu estado interno.

Mensagens são enviadas de uma instância a outro com a finalidade de que a instância receptora produza algum resultado desejado.

A natureza das operações que a instância receptora executa para produzi-lo é ela quem determina, e poderá provocar alterações em seu estado interno, o envio de novas mensagens para outras instâncias e mesmo para si própria, e até a criação de novas instâncias.

Parâmetros podem acompanhar as mensagens dirigidas a uma instância. Tais parâmetros, por sua vez, também são instâncias, e poderão ter algum efeito sobre as operações que a instância receptora executará quando receber a mensagem que elas acompanham.

A descrição das operações que uma instância executa quando recebe uma mensagem é chamada método. O conceito de método é muito parecido com o conceito de procedimento.

A relação que existe entre mensagens e métodos em uma instância é sempre biunívoca, i.e., uma mesma mensagem somente poderia resultar em métodos diferentes quando enviada para instâncias diferentes.

A associação entre mensagens e métodos é sempre pré-definida e estática, i.e., não pode ser alterada em tempo de execução.

Classes

Uma classe é um modelo de instância, i.e., consiste de descrições de estado, métodos, e associações entre mensagens e métodos, que todas as instâncias pertencentes àquela classe irão possuir.

O conceito de classe é muito parecido com o conceito de tipo abstrato de dados, na medida que uma classe define uma estrutura interna e um conjunto de operações que todas as instâncias daquela classe irão possuir.

Classes também podem receber mensagens, ter métodos e ter um estado interno.

Objetos

Todo objeto tem uma classe e serve ao propósito de armazenar instâncias da classe à qual pertence.

Instâncias podem ser criadas a partir de uma classe e, no ato de sua criação, ser solicitadas através de uma mensagem a prestar algum serviço, o que será feito pela execução do método associado àquela mensagem.

Instâncias somente podem ser solicitadas através de uma mensagem a prestar algum serviço, em um momento posterior àquele de sua criação, quando estiver armazenada em um objeto, que lhe provê, com seu nome, uma forma de ser referenciada.

Assim sendo, se não estiverem armazenadas em um objeto, as instâncias tem uma sobrevida breve, já que perdem a utilidade logo após sua criação.

Membros de Classe e de Instância

Todos os dados e métodos definidos em uma classe podem ser ditos membros daquela classe. Podemos classificar os membros de uma classe em (1) dados e métodos de classe; e (2) dados e métodos de instância.

Entendemos que membros de classe, sejam eles dados ou métodos, são membros relativos à uma classe como um todo e não a nenhuma instância individual da classe. Membros de classe expressam propriedades e ações aplicáveis a toda uma classe de instâncias e não a uma instância específica.

Entendemos que membros de instância de classe, sejam eles dados ou métodos, são membros relativos à instâncias individuais e não à classe como um todo. Membros de instância expressam propriedades e ações que são aplicáveis às instâncias de uma classe individualmente e não à classe de modo geral.

Encapsulamento

O conceito de encapsulamento de certa forma resume tudo o que foi dito até agora sobre o modelo de computação proposto pelo paradigma.

Ele determina que uma instância pode ser vista como o encapsulamento de seu estado interno e de seu comportamento. O mesmo ocorre com uma classe.

Herança

O conceito de herança se baseia em uma estrutura hierárquica de classes. Em tal estrutura de classes, cada classe pode ter zero ou mais subclasses e zero ou mais superclasses.

Uma dada subclasse herda todos os componentes (representação de estado interno e comportamento) de suas superclasses. A classe, em adição às propriedades que herda, pode definir componentes próprios, além de poder redefinir componentes que recebeu por herança.

Vale observar que os componentes de uma subclasse recebidos por herança que não forem redefinidos, comportar-se-ão exatamente como componentes definidos na própria classe, e isto sem a necessidade de nenhuma definição ou indicação especial.

Esta propriedade encoraja a definição de novas classes, sem no entanto, requerer uma extensiva duplicação de código [Dershem90].

No paradigma imperativo, o conceito mais próximo de herança é o de subtipos. Da mesma forma que uma subclasse herda as propriedades de suas superclasses, um subtipo herda as de seu supertipo.

O conceito de subtipo, no entanto, é muito mais limitado do que o conceito de herança, porque é aplicável somente a um conjunto limitado de tipos básicos, em geral, aos tipos escalares.

O paradigma imperativo não generaliza esta definição para contemplar tipos abstratos de dados, o que faz do conceito de herança uma grande contribuição do paradigma de orientação a instâncias.

Polimorfismo

Polimorfismo é a propriedade que possibilita que se envie uma mesma mensagem a diferentes instâncias, provocando em cada uma delas uma operação diferente.

Isto acontece porque cada uma das diversas instâncias receptoras potenciais de uma determinada mensagem possui um método próprio para responder à chegada daquela mensagem, dependendo da classe à qual ela pertence.

É importante ressaltar que, ao se enviar uma mensagem, não há a necessidade de se saber nem a classe da instância receptora, nem a forma que esta utilizará para responder à mesma.

No paradigma imperativo, o conceito mais próximo de polimorfismo é o de sobrecarga de operadores e de procedimento.

O conceito de polimorfismo estabelece que a determinação do método a ser invocado, quando da recepção de uma mensagem por uma instância, é feita com base na classe à qual a instância receptora pertence.

Sobrecarga

O conceito de sobrecarga estabelece a possibilidade de se definir 2 ou mais funções com o mesmo nome de forma que, quando ocorrer a chamada de uma função, dentre as diversas funções de mesmo nome, vai-se determinar aquele que será executado pelo número, tipo e ordem dos parâmetros reais fornecidos por ocasião de sua chamada.

A diferença entre este conceito e o conceito de polimorfismo reside no tempo em que é feita a associação do procedimento a ser executado com a chamada de procedimento.

No polimorfismo, a associação ocorre dinamicamente, uma vez que a classe à qual um instância pertence não é conhecida senão em tempo de execução. Na sobrecarga, tudo se passa como se o número e o tipo dos parâmetros constituíssem uma extensão do nome do procedimento, e a associação pode ser estática.

Assinatura

O nome de um método, juntamente com o conjunto ordenados dos tipos dos parâmetros que ele recebe, costuma ser chamado de assinatura .

Herança Múltipla

Herança Múltipla é um recurso previsto pelo paradigma de orientação a objetos cuja finalidade é permitir que classe herde características não de uma, mas de várias outras classes.

Embora não seja um recurso do qual se precise lançar mão com tanta freqüência, trata-se de um recurso que pode por vezes ser muito útil.

Por exemplo: mesmo dispondo de um conjunto amplo de classes base desenvolvidas, pode ser que, num dado momento, nos vejamos confrontados com a necessidade de um novo tipo de objeto.

Pode ser ainda que este novo tipo de objeto guarde muita semelhança, não com uma, mas com muitas das classes que já temos. Trata-se de um caso típico a ser resolvido com herança múltipla.

Quando uma classe é derivada de mais de uma classe, ela pode ser usada em todos os lugares onde for esperada uma de suas classes base. Trata-se de uma extensão do conceito de polimorfismo.

Classes Abstratas

Numa situação convencional de herança, tem-se que classes bases compartilham com suas classes derivadas, tanto a seu comportamento, quanto sua implementação.

Entretanto, podem haver situações nas quais quase não exista, de fato, o compartilhamento de implementação, embora haja um efetivo compartilhamento de comportamento.

Geralmente não é possível escrever código para muitos dos métodos dessas classes, que, apesar de declarados, não são de fato definidos, existindo sem um corpo onde ocorra sua implementação.

Chamamos este tipo de método de Métodos Abstratos e classes que contem um ou mais métodos abstratos são chamadas de Classes Abstratas.

Não é permitido criar instâncias de classes abstratas, mas, apesar disso, elas podem ser muito úteis por propiciarem o polimorfismo.

Conclusão

As linguagens orientadas a objetos nos apresentam uma nova forma de organizar e estruturar programas e isto leva à necessidade de desenvolver uma nova forma de pensar em resolução computacional de problemas.

O Paradigma de Orientação a Objetos conduz naturalmente à implementação de módulos fortemente coesos e fracamente acoplados. Isto torna os programas orientados a objetos mais manuteníveis e suas partes mais reusáveis, facilitando o que em Engenharia de Software chamamos de Peça de Software.

Por tudo isso, cada vez mais encontramos o Paradigma de Orientação a Objetos incorporado nos mais diversos ambientes de computação de uso contemporâneo.

Anexo I

Referências

Dershem, H.L.; and Jipping, M.J., “Programming Languages: Structures and Models”, Wadsworth, Inc., 1990.

Takahashi, T., “Introdução à Programação Orientada a Objetos”, III EBAI - Escola Brasileiro-Argentina de Informática, 1988.

Prof. André Reis
dos
Carvalho
Gomes

Anexo II

Bibliografia

Sebesta, R.W., “Concepts of Programming Languages”, The Benjamin/Cummings Publishing Company, Inc., 1989.

Ghezzi, Java., Jazayeri, M., “Programming Languages Concepts”, John Wiley & Sons, Inc., 1982.

Dershem, H.L.; and Jipping, M.J., “Programming Languages: Structures and Models”, Wadsworth, Inc., 1990.

Pratt, T.W., Programming Languages: Design and Implementation.

Takahashi T.; e Liesemberg, H.K.; "Programação Orientada a Objetos"

Entsminger, G.; "The TAO of Objects: A Beginner's Guide to Object Oriented Programming"

Cox, B.J., Programação Orientada para Instância. Morrison, M.; December, John; et alii, “Java Unleashed”, Sams.net Publishing, 1996.

Richardson, J.E.; Schulz, R.C.; e Berard, E.V.; "A Complete Object Oriented Design Example"